Grado Superior

Programación

T4. Introducción a Java

Curso 2025/26

Profesor

Raúl Murillo Gallego

«La lógica te llevará de la A a la Z. La imaginación te llevará a cualquier lugar» —Albert Einstein

Presentación

4.1. Java
- 4.1.1. Descripción de la tecnología
- 4.1.2. Versiones de Java
- 4.1.3. Requisitos previos
- 4.1.4. Máquina virtual JVM
- 4.1.5. Entorno de desarrollo JDK
- 4.1.6. Entorno de ejecución JRE
- 4.1.7. Variables de entorno PATH y CLASSPATH
- 4.1.8. Tipos de archivos empleados
- 4.1.9. Recolector de basura

4.2. Entorno de Desarrollo Integrado
- 4.2.1. Características y elementos
- 4.2.2. Alternativas
- 4.2.3. Configuración
- 4.2.4. Manejo del IDE

4.3. Elementos de básicos de programación en Java
- 4.3.1. Tipo de datos
- 4.3.2. Operadores
- 4.3.3. Java y las matemáticas

4.4. Estructuras de control en Java
- 4.4.1. Estructuras selectivas
- 4.4.2. Estructuras repetitivas

T4. Introducción a Java

4.1. Java
- **4.1.1. Descripción de la tecnología**
- **4.1.2. Versiones de Java**
- **4.1.3. Requisitos previos**
- **4.1.4. Máquina virtual JVM**
- **4.1.5. Entorno de desarrollo JDK**
- **4.1.6. Entorno de ejecución JRE**
- **4.1.7. Variables de entorno PATH y CLASSPATH**
- **4.1.8. Tipos de archivos empleados**
- **4.1.9. Recolector de basura**

4.2. Entornos de Desarrollo Integrado
- 4.2.1. Características y elementos
- 4.2.2. Alternativas
- 4.2.3. Configuración
- 4.2.4. Manejo del IDE

4.3. Elementos de básicos de programación en Java
- 4.3.1. Tipo de datos
- 4.3.2. Operadores
- 4.3.3. Java y las matemáticas

4.4. Estructuras de control en Java
- 4.4.1. Estructuras selectivas
- 4.4.2. Estructuras repetitivas

4.1. Java

Descripción de la tecnología

Java es uno de los lenguajes de programación más populares, que se pueden utilizar libremente y que cuenta con una alta demanda en el mercado según Índice TIOBE. Este potente lenguaje permite programar casi cualquier tipo de aplicación (juegos, aplicaciones web, aplicaciones de escritorio, servicios…) y está compuesto de varias tecnologías.

Java fue anunciado públicamente el 23 de mayo de 1995, pero no fue hasta el 23 de enero del 1996 cuando se publicó la primera versión del JDK (*JDK 1.0*) de la mano de *Sun Microsystems*, adquirida (20 de abril de 2009) por *Oracle,* con la intención de facilitar el desarrollo de programas sin tener en cuenta el hardware final ni los sistemas operativos sobre el que se ejecutaría.

Versiones de Java

- J2ME (Java 2 Micro Edition), es la distribución destinada al desarrollo del software para dispositivos de recursos más limitados (electrodomésticos, TDT, impresoras, teléfonos móviles, PDA). Tuvo mucho éxito debido al auge de los juegos en teléfonos móviles, aunque su uso ha decrecido notablemente.

- J2SE (Java 2 Standard Edition), es la distribución destinada al desarrollo de aplicaciones de escritorio para ordenadores personales o servidores, así como al desarrollo de aplicaciones cliente-servidor y conexiones con bases de datos (JDBC)

- J2EE (Java 2 Enterprise Edition), es la distribución más completa de las tres y expande la funcionalidad de J2SE con soporte y API para aplicaciones basada en tecnologías web como , por ejemplo, WebServices.

Requisitos para poder programar en Java

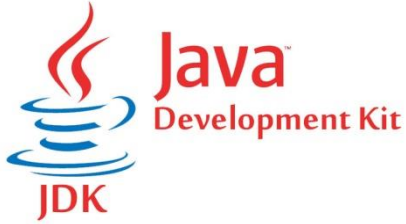
¿Qué necesitamos para poder programar en Java?

- **JVM** (Java Virtual Machine)

- **JRE** (Java Runtime Environment)

- **JDK** (Java Development Kit)

- **IDE** (IntelliJ IDEA Ultimate)



JVM (Java Virtual Machine)

- **Definición:** Es la máquina virtual que ejecuta los programas Java. La JVM interpreta y ejecuta el bytecode de Java, que es el código intermedio generado tras la compilación. La JVM es lo que hace que Java sea multiplataforma, ya que el mismo bytecode puede ejecutarse en cualquier sistema operativo que tenga una JVM compatible.

- **Función principal**: Ejecutar programas Java.

JRE (Java Runtime Environment)

- **Definición:** Es el entorno necesario para ejecutar aplicaciones Java. Incluye la JVM junto con las bibliotecas y archivos esenciales que un programa Java necesita para correr. Sin embargo, el JRE no incluye herramientas para desarrollar o compilar código, solo para ejecutarlo. Actualmente estamos en la versión **Java 8 Update 461**.

- **Función principal:** Permitir la ejecución de aplicaciones Java en un sistema.

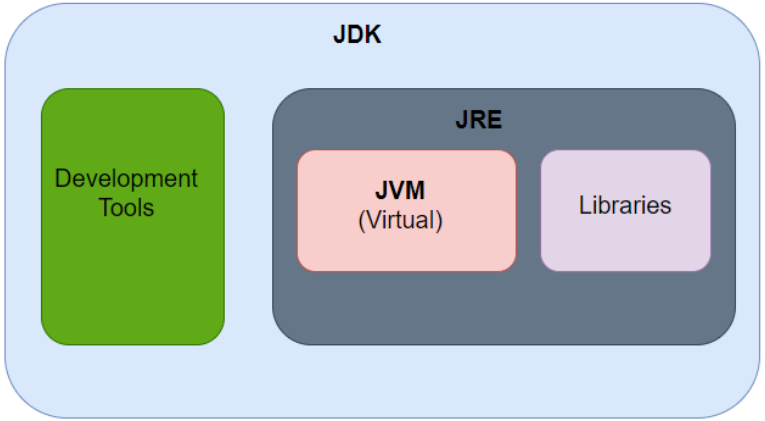
JDK (Java Development Kit)

- **Definición:** Es el kit de desarrollo para Java, que incluye el JRE, la JVM, y además herramientas para desarrollar aplicaciones en Java, como el compilador javac, depuradores, y otras utilidades necesarias para la programación. El JDK es necesario para escribir, compilar y ejecutar programas Java. Actualmente estamos en la versión 23 (**JDK 23**).

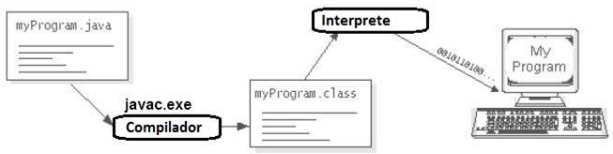
- **Función principal:** Proporcionar todo lo necesario para desarrollar y compilar aplicaciones Java.

Resumen

- **JVM:** Ejecuta bytecode de Java.
- **JRE:** Entorno para ejecutar programas Java (incluye la JVM).
- **JDK:** Herramientas para desarrollar y compilar programas Java (incluye el JRE y la JVM).



Proceso de compilación en Java

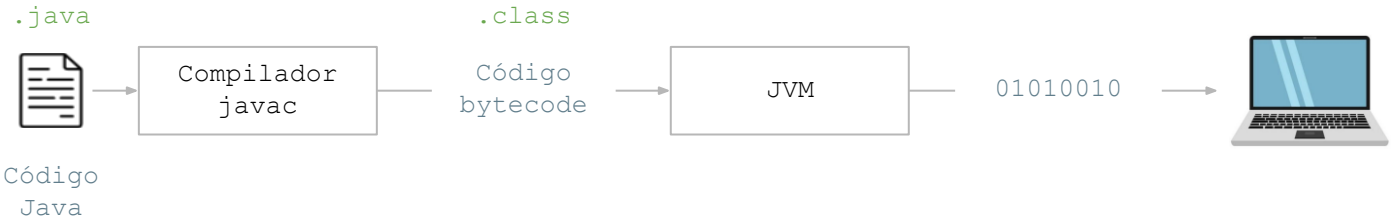


1º Compilador (`javac`):

- Cuando escribes código Java, primero lo pasas por un **compilador** que convierte el código fuente en **bytecode**. Este bytecode es un formato intermedio que no es directamente ejecutable por el sistema operativo, pero puede ser interpretado por la máquina virtual de Java (JVM).
- El compilador de Java transforma los archivos `.java` en archivos `.class`, que contienen el bytecode.

2º JVM (Java Virtual Machine):

- Una vez compilado, el bytecode es ejecutado por la **JVM**, que actúa como un **intérprete**. La JVM traduce el bytecode a instrucciones que el sistema operativo y el hardware pueden entender y ejecutar.



Variables de entorno PATH y CLASSPATH

- **PATH:** Es la variable del sistema utilizada por el sistema operativo para buscar los ejecutables de las diferentes aplicaciones.

- **CLASSPATH:** Es una variable de entorno que Java utiliza para especificar la ubicación de clases y paquetes necesarios para ejecutar una aplicación. Se utiliza para indicar a la JVM y al compilador (javac) dónde buscar archivos .class (clases compiladas) y bibliotecas externas (archivos .jar).

El uso de esta variable ya que IDEs como Eclipse, IntelliJ IDEA y NetBeans también gestionan el CLASSPATH de forma automática, facilitando la vida del desarrollador.

Tipos de archivos empleados I

1. Archivos `.java`

- **Descripción**: Son los archivos de código fuente que contienen el código escrito por el desarrollador en el lenguaje Java.
- **Uso**: Contienen las clases, interfaces, métodos, y todo el código fuente que será compilado.
- **Ejemplo**: `MiClase.java`

2. Archivos `.class`

- **Descripción**: Son los archivos generados por el **compilador de Java** (javac) cuando se compila un archivo `.java`. Estos archivos contienen el **bytecode** que será interpretado y ejecutado por la JVM.
- **Uso**: La JVM utiliza estos archivos para ejecutar el programa.
- **Ejemplo**: `MiClase.class`

Tipos de archivos empleados II

3. Archivos `.jar` (Java Archive)

- **Descripción**: Son archivos que contienen múltiples archivos .class, recursos (como imágenes o archivos de configuración) y metadatos empaquetados en un formato comprimido **ZIP**. Normalmente incluyen un archivo de manifiesto (MANIFEST.MF) que describe las propiedades del archivo JAR, como la clase principal para ejecutar.
- **Uso**: Se utilizan para distribuir bibliotecas, aplicaciones Java o componentes de código reutilizable. También se usan para empaquetar aplicaciones completas.
- **Ejemplo**: `miAplicacion.jar`

Recolector de basura

En Java, el **recolector de basura** (Garbage Collector) es un proceso automático que se encarga de gestionar la memoria en la JVM (Java Virtual Machine), liberando el espacio ocupado por objetos que ya no son utilizados por el programa, es decir, aquellos que ya no tienen referencias activas. El objetivo del recolector de basura es evitar fugas de memoria y asegurar que la memoria se utiliza de manera eficiente.

Ventajas: Facilita el desarrollo y evita errores humanos a la hora de gestionar la memoria. Gracias a esta liberación, es posible escribir código con la tranquilidad de los programas desarrollados no terminarán consumiendo los recursos del sistema antes de que el programa finalice.

Desventajas: El proceso de recolección consume CPU y memoria, y su impacto varía según el recolector utilizado. Es decir, este elemento consume recursos del sistema.

T4. Introducción a Java

4.1. Java
- 4.1.1. Descripción de la tecnología
- 4.1.2. Versiones de Java
- 4.1.3. Requisitos previos
- 4.1.4. Máquina virtual JVM
- 4.1.5. Entorno de desarrollo JDK
- 4.1.6. Entorno de ejecución JRE
- 4.1.7. Variables de entorno PATH y CLASSPATH
- 4.1.8. Tipos de archivos empleados
- 4.1.9. Recolector de basura

4.2. Entorno de Desarrollo Integrado
- **4.2.1. Características**
- **4.2.2. Comparativa IDE para Java**
- **4.2.3. Configuración**
- **4.2.4. Manejo del IDE**

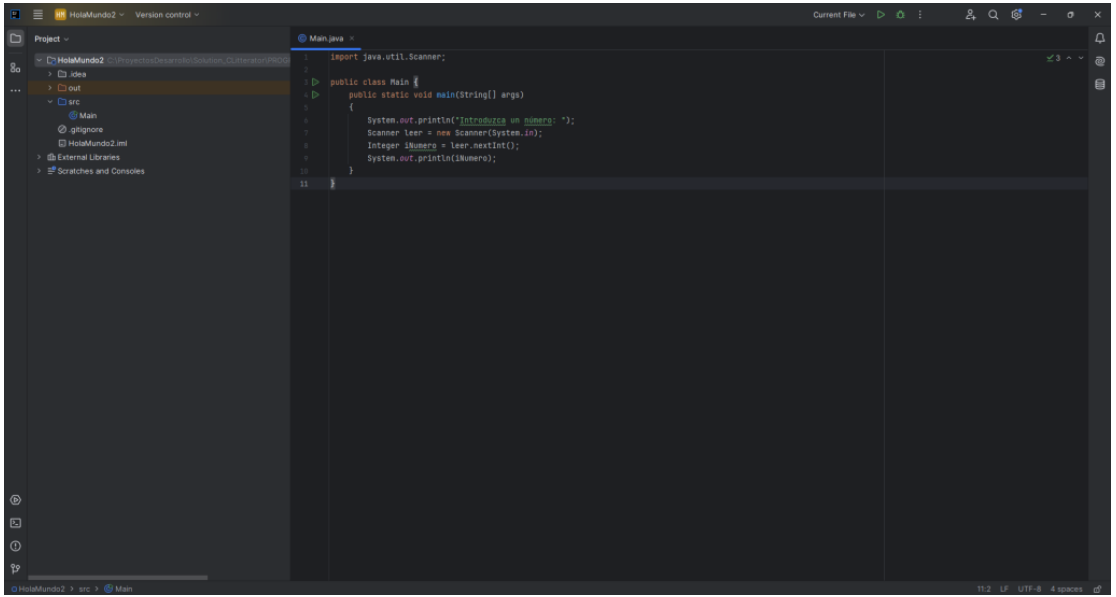
4.3. Elementos de básicos de programación en Java
- 4.3.1. Tipo de datos
- 4.3.2. Operadores
- 4.3.3. Java y las matemáticas

4.4. Estructuras de control en Java
- 4.4.1. Estructuras selectivas
- 4.4.2. Estructuras repetitivas

4.2. Entorno de Desarrollo Integrado

Introducción

Un entorno de desarrollo integrado (IDE, por sus siglas en inglés: Integrated Development Environment) es una herramienta que proporciona un conjunto de características que permiten a los desarrolladores escribir, editar, probar y depurar código de manera eficiente. Los IDE son fundamentales para aumentar la productividad y reducir errores.



Características IDE

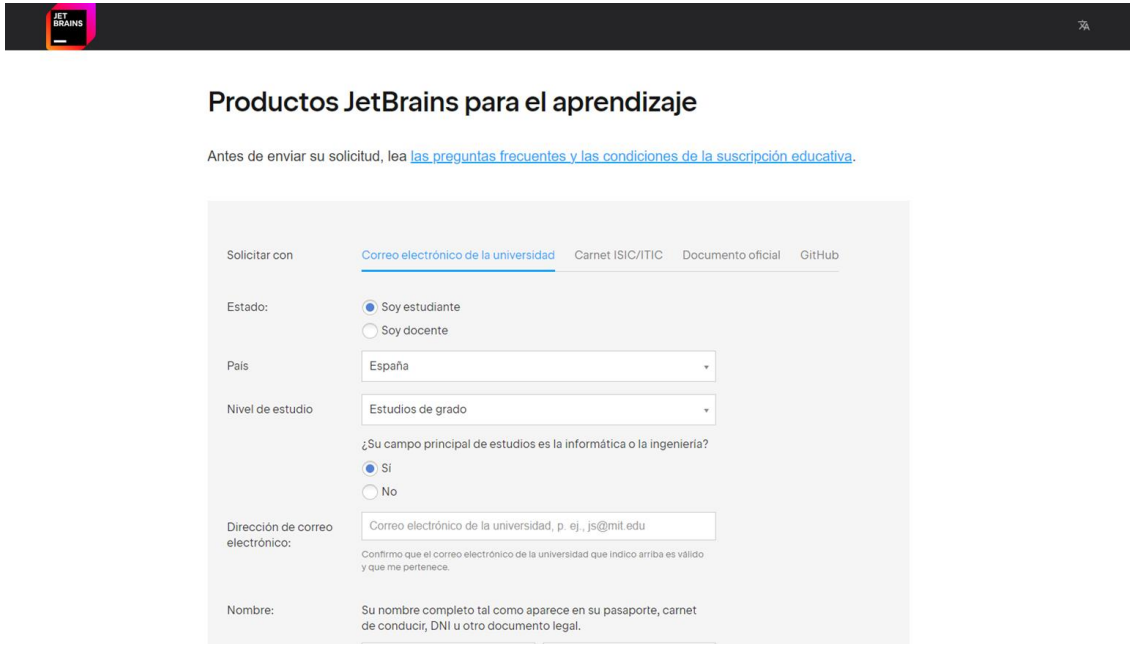
- **Editor de código fuente**: Es el componente más básico y esencial de un IDE. Permite escribir y editar el código.
- **Depurador (Debugger)**: Herramienta que permite la ejecución paso a paso del código, identificación de errores (bugs) y análisis detallado de cómo se comporta el programa durante su ejecución.
- **Compilador/Intérprete**: Herramienta integrada que traduce el código fuente a un lenguaje que la máquina pueda ejecutar. En lenguajes interpretados, el intérprete ejecuta el código línea por línea.
- **Gestor de proyectos:** Los IDE organizan el código en proyectos, permitiendo gestionar múltiples archivos y carpetas asociados con el mismo.
- **Integración con control de versiones:** Muchos IDE se integran con sistemas de control de versiones como Git, Subversion (SVN), o Team Foundation.
- **Terminal o consola:** Proporciona un acceso directo a una consola dentro del propio IDE, permitiendo ejecutar comandos del sistema operativo o relacionados con el proyecto.
- **Herramientas de prueba (Testing)**:Integración con frameworks de prueba que permiten a los desarrolladores escribir y ejecutar pruebas automáticas (unitarias, de integración, etc.).

Comparativa IDE para Java

Características	IntelliJ IDEA	NetBeans	Eclipse
Lenguajes soportados	Java, Kotlin, Scala, Groovy, Python, PHP, y muchos otros con plugins.	Java, PHP, C/C++, HTML5, JavaScript, entre otros.	Java, C/C++, PHP, JavaScript, Python, y muchos más con plugins.
Facilidad de uso	Muy intuitivo y fácil de usar. Gran enfoque en la usabilidad.	Relativamente fácil de usar, interfaz sencilla.	Puede ser más complejo al principio debido a su gran cantidad de opciones y configuración.
Rendimiento	Alto, con optimizaciones automáticas. Puede consumir más recursos (RAM/CPU).	Moderado. Es ligero pero no tan rápido como IntelliJ.	Puede ser más lento si se cargan demasiados plugins, especialmente en proyectos grandes.
Autocompletado	Excelente, con sugerencias muy inteligentes y predictivas (AI-driven).	Bueno, pero no tan avanzado como IntelliJ.	Bueno, pero algo menos preciso y avanzado que IntelliJ.
Refactorización de código	Herramientas avanzadas de refactorización, fácil de usar y muy potente.	Buena, aunque no tan sofisticada como IntelliJ.	Buena, pero requiere más configuraciones y puede ser menos fluida que en IntelliJ.
Depuración (debugging)	Depurador muy completo, permite debugging remoto, pruebas y perfilado.	Herramientas de depuración decentes, aunque menos avanzadas.	Herramientas de depuración sólidas, con opciones avanzadas pero algo más complicadas de usar.
Plugins y extensiones	Gran ecosistema de plugins, fácil instalación desde la tienda de JetBrains.	Soporte para plugins, pero no tan extenso como IntelliJ.	Muy extensible, enorme repositorio de plugins, aunque algunos son difíciles de configurar.
Licencia	Versión gratuita (Community) y de pago (Ultimate con más características).	Totalmente gratuito, open-source.	Gratuito, open-source. Algunas versiones premium con características adicionales (Ej: MyEclipse).
Soporte para control de versiones	Excelente integración con Git, GitHub, Mercurial, y otros sistemas.	Buena integración con Git, Subversion y otros.	Excelente integración con Git, Subversion, Mercurial, y otros.
Documentación y comunidad	Documentación oficial muy detallada y comunidad activa.	Documentación adecuada, comunidad activa en foros.	Amplia documentación y comunidad, pero a veces es difícil encontrar soluciones específicas.
Uso en proyectos grandes	Ideal para proyectos de gran escala. Muy eficiente con refactorización y gestión de grandes bases de código.	Puede funcionar bien con proyectos grandes, pero puede ralentizarse.	Apto para proyectos grandes, pero la performance puede degradarse con muchos plugins instalados.
Actualizaciones	Frecuentes, con nuevas características y mejoras constantes.	Actualizaciones menos frecuentes, pero el soporte sigue activo.	Frecuentes, con soporte continuo de la comunidad.
Entorno de desarrollo web	Excelente, con soporte nativo para muchos frameworks web (Angular, React, Spring, etc.).	Soporte decente, pero menos intuitivo que en IntelliJ.	Muy bueno, especialmente con la ayuda de plugins, pero requiere más configuración inicial.

Configuración / Instalación

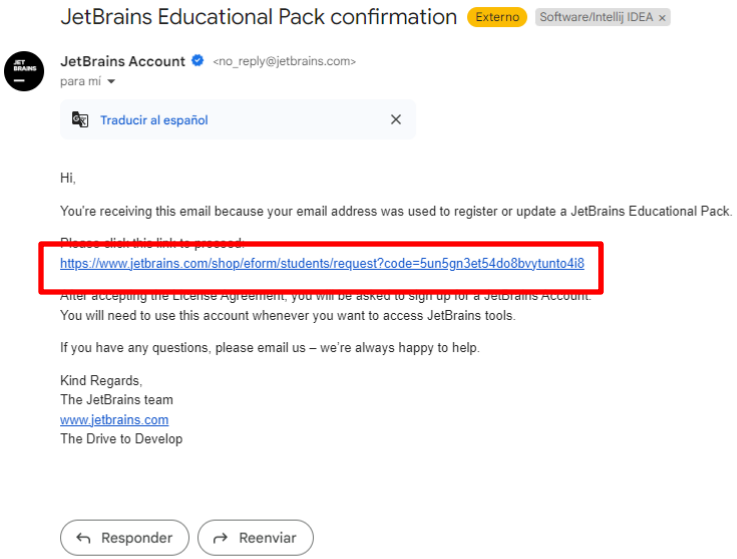
Paso 1: Hay que darse de alta en el siguiente enlace: https://www.jetbrains.com/shop/eform/students



Importante: Guardar el usuario, password y el id de la licencia

Configuración / Instalación

Paso 2: Ir a nuestro correo del colegio nombre.apellido@colegiolitterator.com y pinchar en el enlace que aparece



JetBrains Educational Pack confirmation Externo Software/Intellij IDEA ×

JetBrains Account <no_reply@jetbrains.com>
para mí

Traducir al español

Hi,

You're receiving this email because your email address was used to register or update a JetBrains Educational Pack.

Please click this link to proceed:
https://www.jetbrains.com/shop/eform/students/request?code=5un5gn3et54do8bvytunto4i8

After accepting the License Agreement, you will be asked to sign up for a JetBrains Account.
You will need to use this account whenever you want to access JetBrains tools.

If you have any questions, please email us – we're always happy to help.

Kind Regards,
The JetBrains team
www.jetbrains.com
The Drive to Develop

Responder Reenviar

Configuración / Instalación

Paso 3: Debemos recibir un correo parecido a este:

License Certificate for your JetBrains Educational Pack / Order D381200232 Externo Software/Intellij IDEA ×

JetBrains Sales <sales@jetbrains.com> mié, 25 sept, 18:19
para mí

Traducir al español ×

Hi Raúl,

Congratulations! Your JetBrains Educational Pack has been confirmed.

Please link your free license to a new or an existing JetBrains Account. You will need to use this account whenever you want to access JetBrains tools.

The link above expires on September 29, 2024.

To access your license after this date, go to the JetBrains Account portal and sign in or register a new account with the email address at which you received this message.

Here are some useful materials to go with your Educational Pack:

- The JetBrains Academy lets you create courses with coding tasks and custom integrated tests inside JetBrains IDEs.
- The JetBrains Academy on Hyperskill allows students to build interactive projects. Your students can study for free or receive a 50% discount on a Premium subscription for 3 months with the coupon code **EDUPACK50**.
- The IDE Features Trainer plugin allows users to learn basic shortcuts and essential features from right inside IntelliJ IDEA and other IDEs.

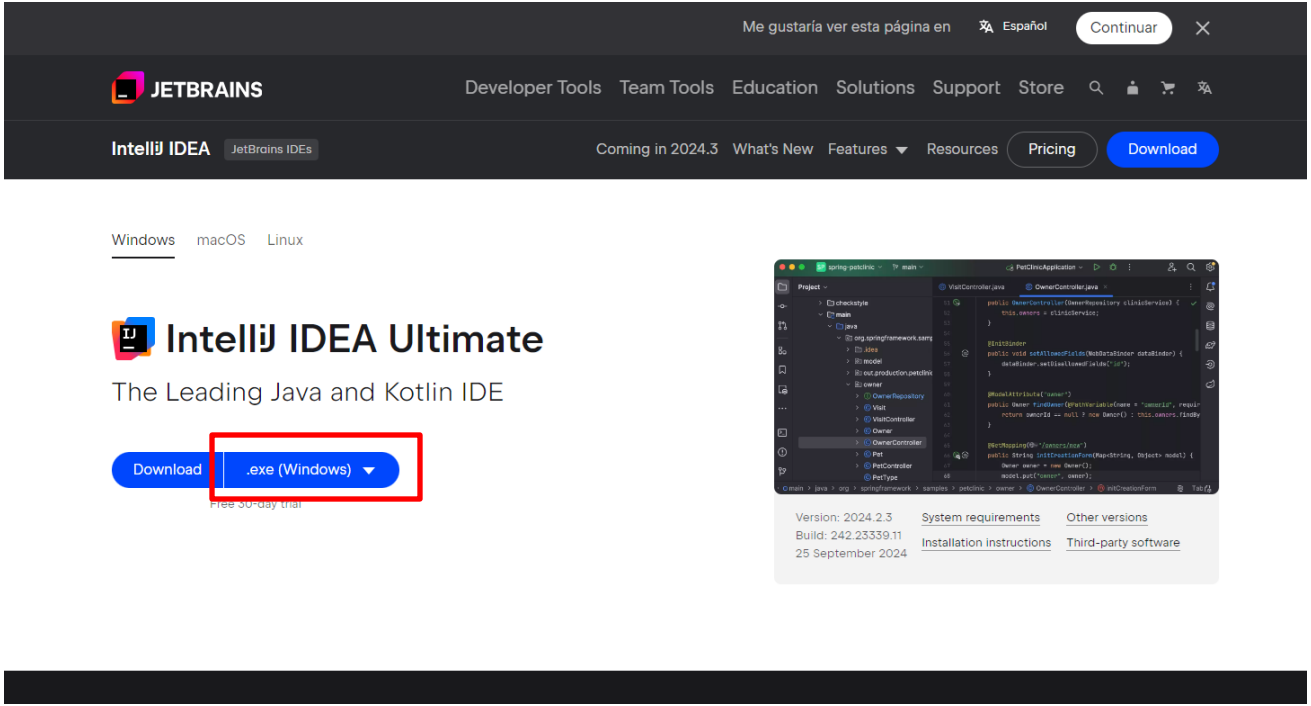
If you have any questions, please email us and we will be glad to help.

Kind Regards,
The JetBrains team

www.jetbrains.com
The Drive to Develop

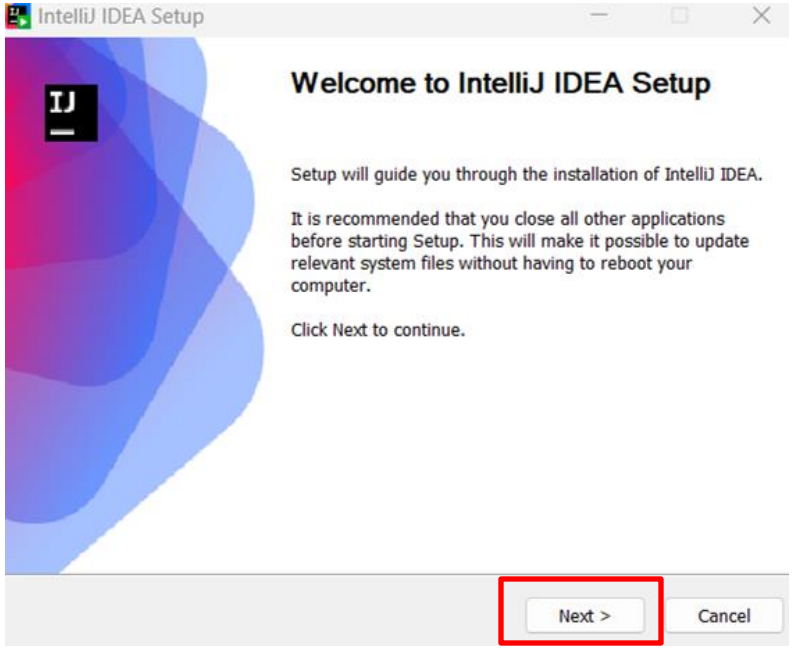
Configuración / Instalación

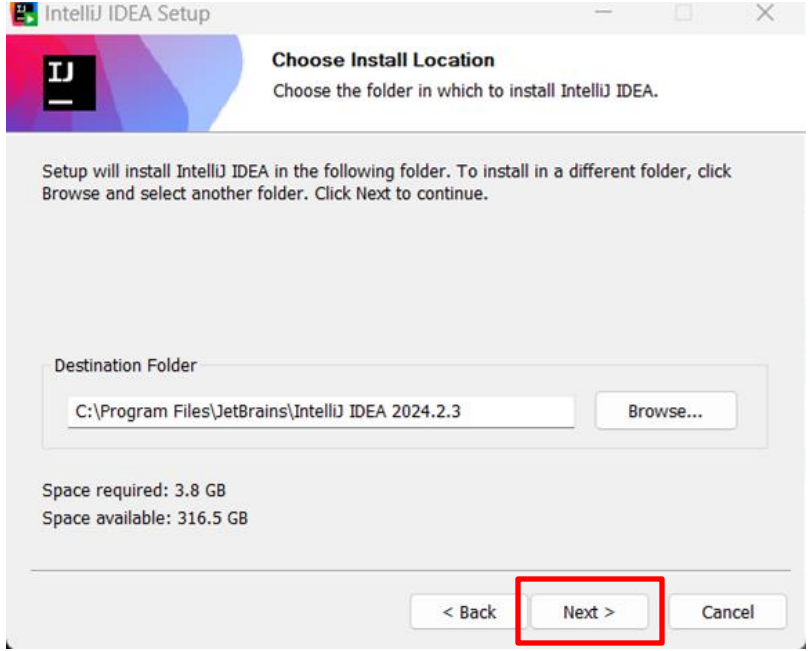
Paso 4: Descargar el archivo de la siguiente web: https://www.jetbrains.com/idea/download/?section=windows



Configuración / Instalación

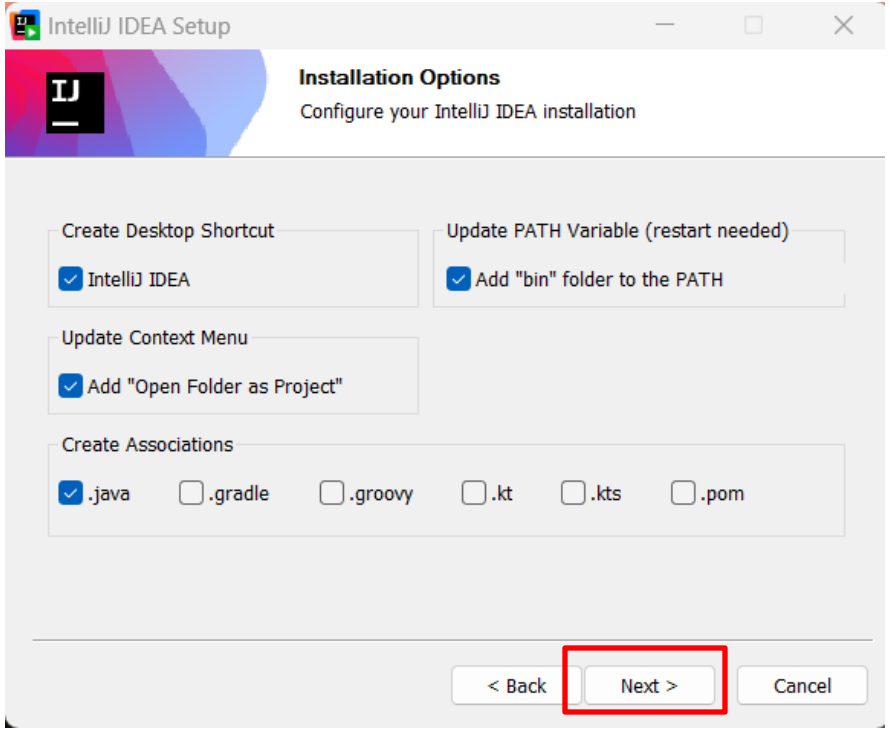
Paso 5: Pinchar en el .exe descargado. Debe aparecer la siguiente pantalla

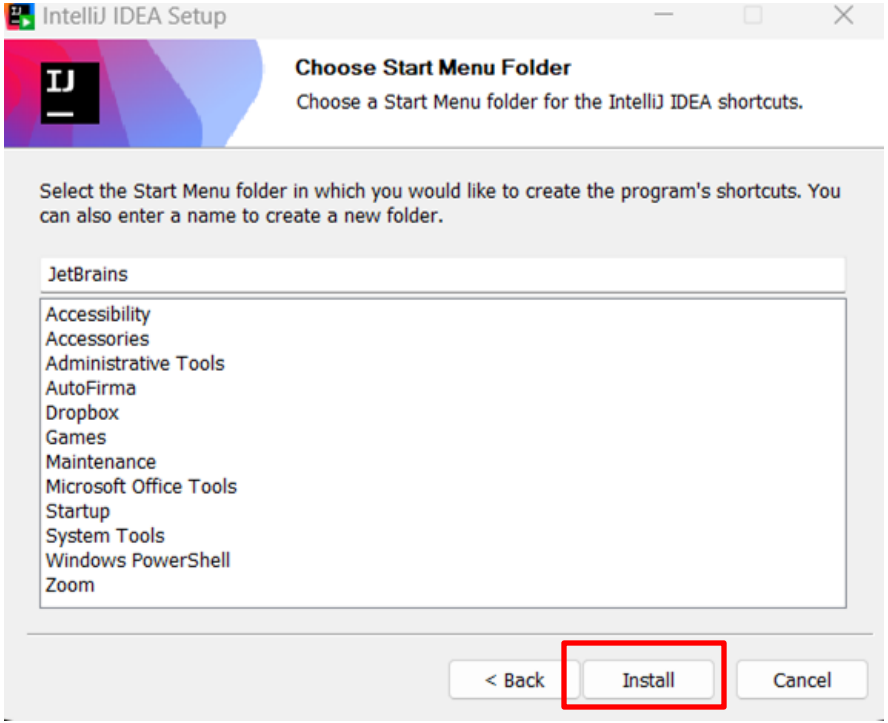




Configuración / Instalación

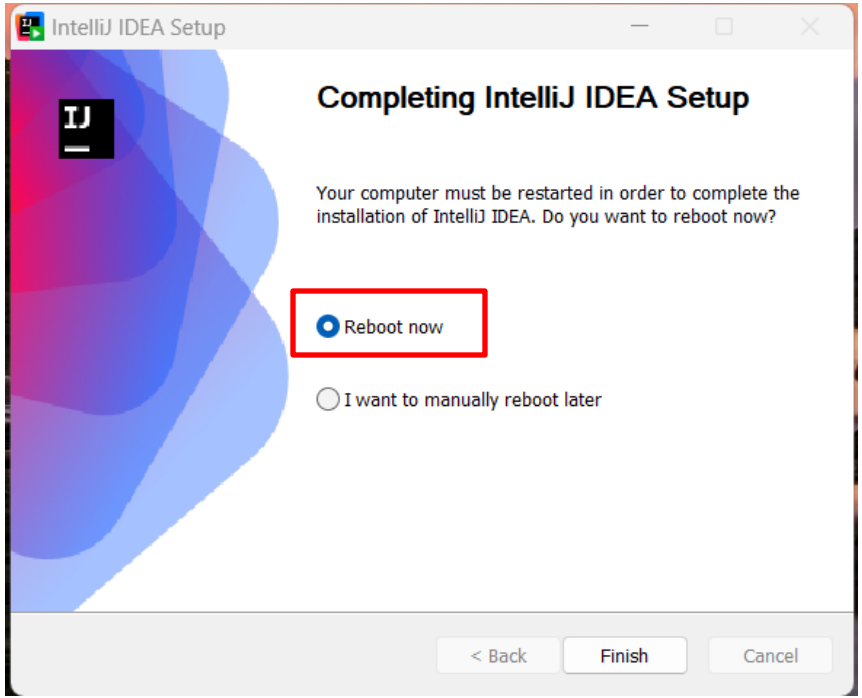
Paso 6: En esta pantalla debemos elegir los siguientes valores





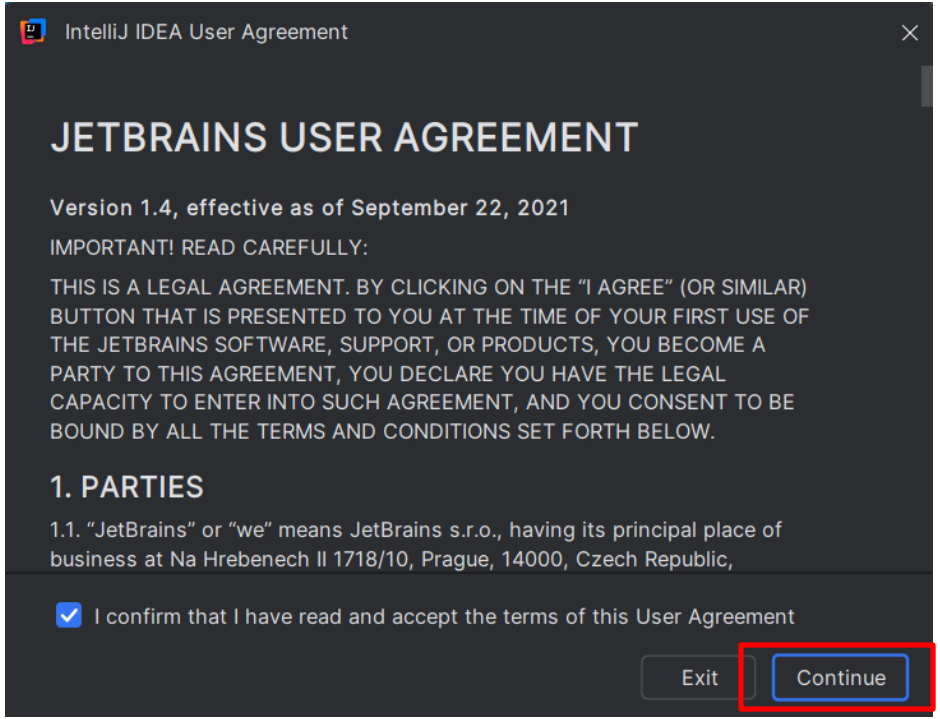
Configuración / Instalación

Paso 7: En esta pantalla debemos de reiniciar nuestro ordenador



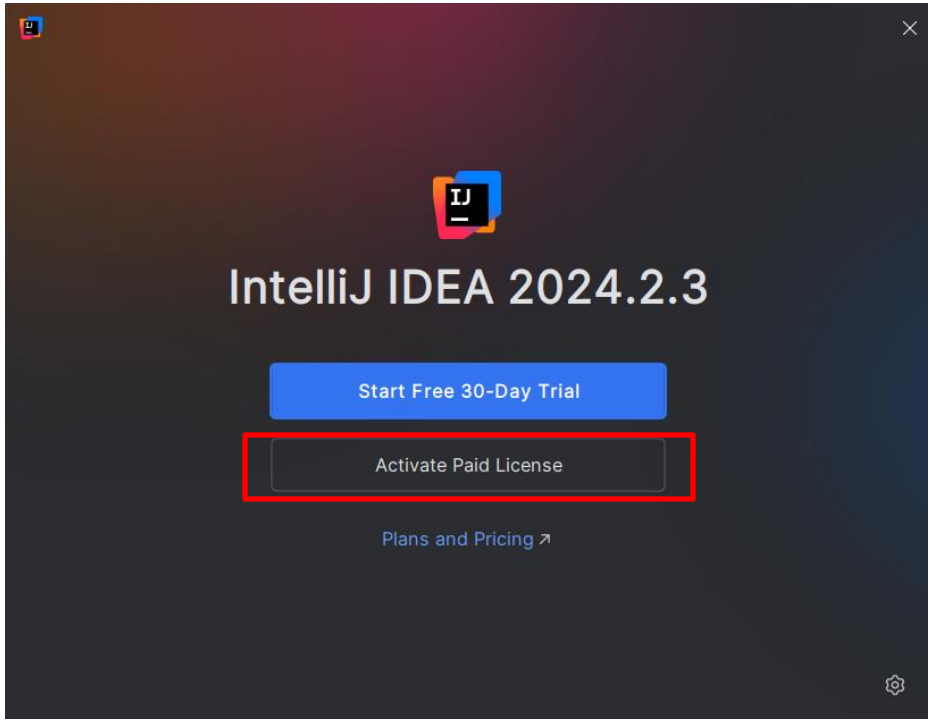
Configuración / Instalación

Paso 8: Una vez reiniciado el ordenador. Pinchamos en en el icono de Intellij IDEA. Y aceptamos las condiciones de uso.



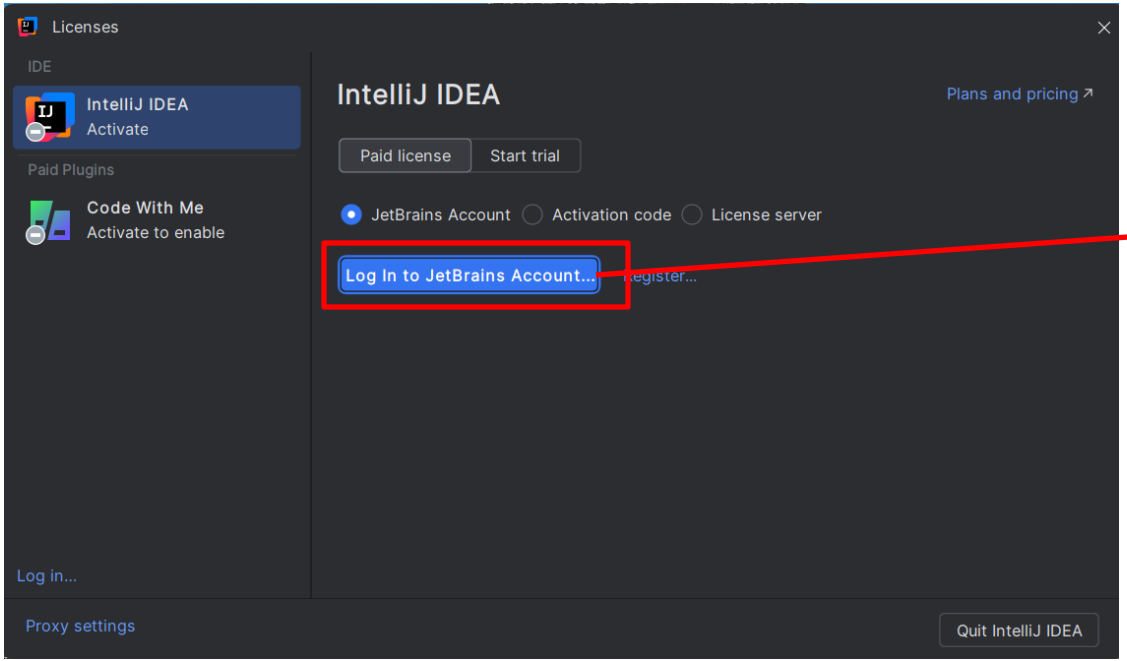
Configuración / Instalación

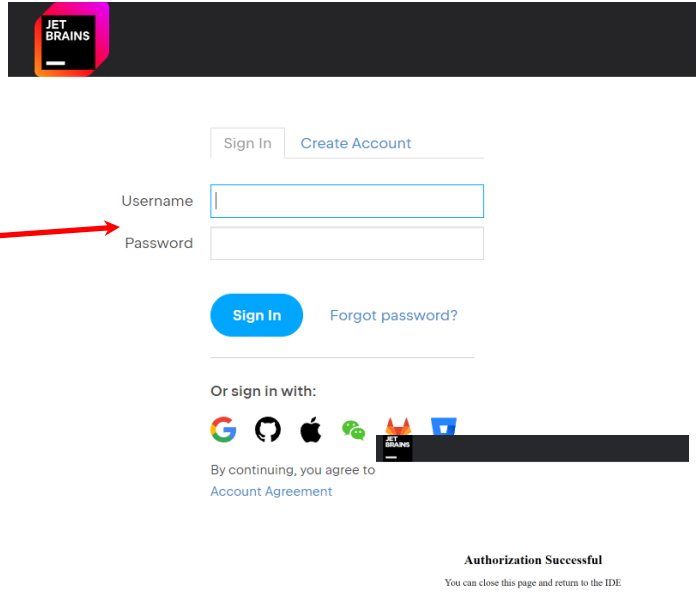
Paso 9: Activamos la licencia del producto Intellij IDEA.



Configuración / Instalación

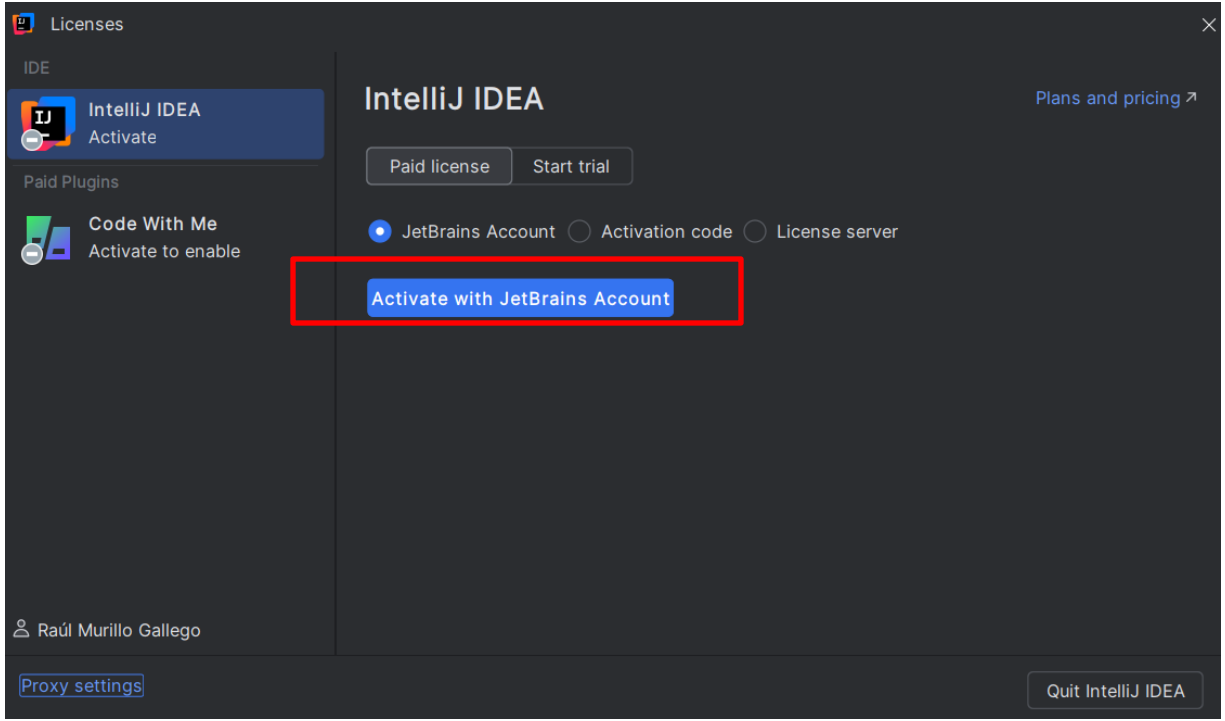
Paso 10: Nos “logueamos” para insertar nuestro usuario y contraseña que previamente hemos registrado





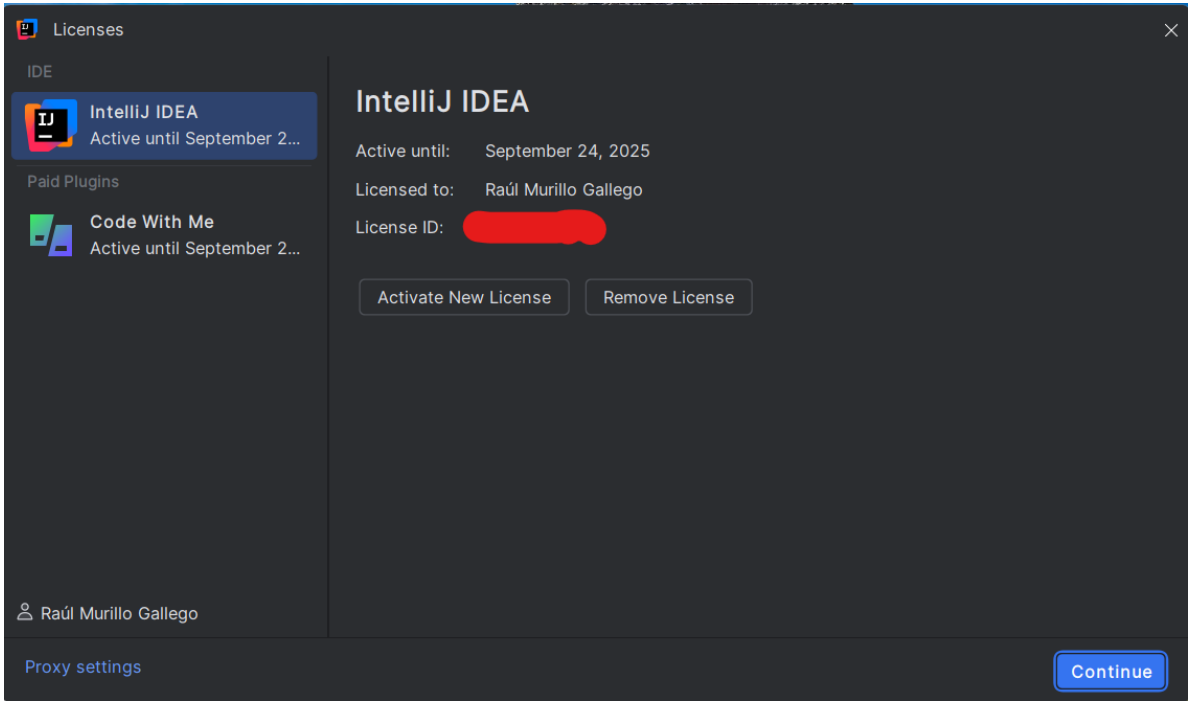
Configuración / Instalación

Paso 11: Activamos la licencia del producto Intellij IDEA.



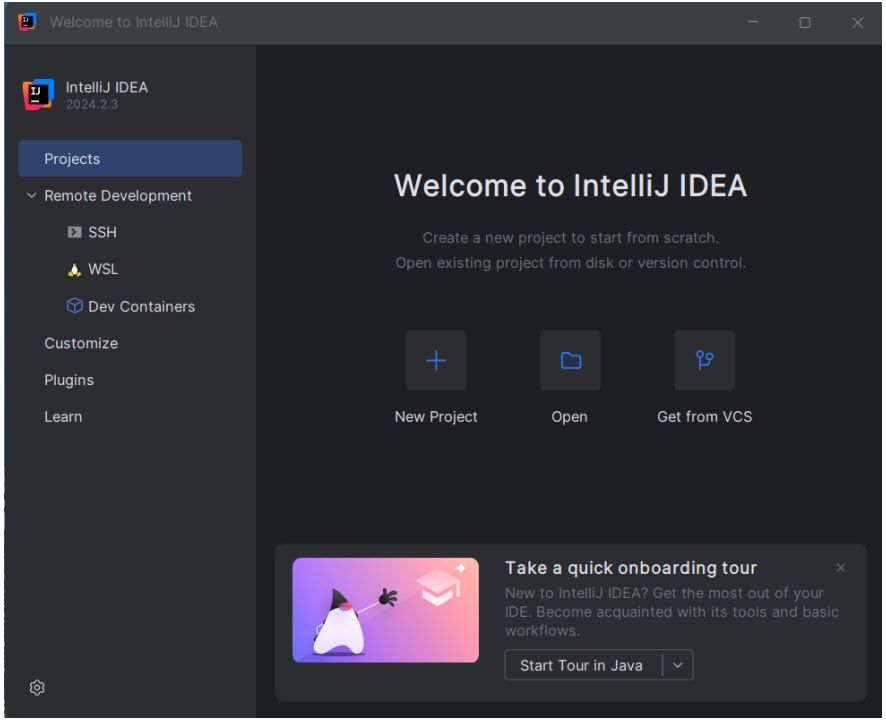
Configuración / Instalación

Paso 12: Una vez activada la licencia, debe aparecer una ventana como la siguiente:



Configuración / Instalación

Paso 13: Y ya tendríamos instalado nuestro IDE con nuestra licencia correspondiente



HolaMundo1 Version control Current File

Project

- HolaMundo1 C:\Temp\Solution_CLitterator\HolaMundo1
  - .idea
  - src
    - Main
  - .gitignore
  - HolaMundo1.iml
- External Libraries
- Scratches and Consoles

Main.java

To Run code, press Mayús F10 or click the icon in the gutter.

```
3   public class Main {  no usages
4       public static void main(String[] args) {
7           System.out.printf("Hello and welcome!");
8
9           for (int i = 1; i <= 5; i++) {
            System.out.println("i = " + i);
13          }
14      }
15  }
```

Press Alt Enter with your caret at the highlighted text to see how IntelliJ IDEA suggests fixing it.

Press Mayús F9 to start debugging your code. We have set one breakpoint for you, but you can always add more by pressing Ctrl F8.

3 1

Microsoft Defender configuration

The IDE has detected Microsoft Defender with Real-Time Protection enabled. It...

Automatically More

HolaMundo1 > src > Main

Installing Oracle OpenJDK 23... 1:1 LF UTF-8 4 spaces

T4. Introducción a Java

4.1. Java
- 4.1.1. Descripción de la tecnología
- 4.1.2. Versiones de Java
- 4.1.3. Requisitos previos
- 4.1.4. Máquina virtual JVM
- 4.1.5. Entorno de desarrollo JDK
- 4.1.6. Entorno de ejecución JRE
- 4.1.7. Variables de entorno PATH y CLASSPATH
- 4.1.8. Tipos de archivos empleados
- 4.1.9. Recolector de basura

4.2. Entorno de Desarrollo Integrado
- 4.2.1. Características
- 4.2.2. Comparativa IDE para Java
- 4.2.3. Configuración
- 4.2.4. Manejo del IDE

4.3. Elementos de básicos de programación en Java
- **4.3.1. Tipo de datos**
- **4.3.2. Operadores**
- **4.3.3. Java y las matemáticas**

4.4. Estructuras de control en Java
- 4.4.1. Estructuras selectivas
- 4.4.2. Estructuras repetitivas

4.3. Elementos de básicos de programación en Java

Tipos de datos en Java

Java es un lenguaje pródigo en tipo de datos, y además es **fuertemente tipado**, es decir, que todo dato que vaya a ser utilizado por la aplicación de ser necesariamente declarado al tipo a que se corresponde.

En una primera aproximación, podemos decir que nos encontramos con datos de tipo :

- **Tipos de datos primitivos.-** Los tipos primitivos son los tipos de datos más básicos que vienen predefinidos en Java.

- **Tipos de datos referenciados (objetos).-** Los tipos referenciados se utilizan para almacenar **objetos**, que son instancias de **clases**. Estos tipos de datos no almacenan directamente el valor, sino una referencia a un objeto en la memoria

Tipos de datos primitivos

Tipo	Longitud	Naturaleza	Rango	Ejemplo
boolean	1 bit	lógico	true/false	boolean esmayordeedad = true;
char	16 bits	carácter	unicode	char inicial = 'a';
byte	8 bits	entero	de -128 a 127	byte edad = 25;
short	16 bits		-32,768 a 32,767	short poblacion = 15000;
int	32 bits		-2,147,483,648 a 2,147,483,647	int salario = 100000;
long	64 bits		-9,223,372,036,854,775,808 a 9,223,372,036,854,775,807	long distancia = 12345678910l; // el sufijo 'l' indica un literal de tipo long
float	32 bits	real	6-7 dígitos decimales	float temperatura = 36.6f; // el sufijo 'f' indica un literal de tipo float.
double	64 bits		15-16 dígitos decimales	double pi = 3.141592653589793;

Tipos de datos referenciados (objetos)

String (cadenas de texto)

Las variables de tipo String almacenan cadenas de caracteres. Aunque en otros lenguajes una cadena es solo una secuencia de caracteres, en Java, String es una clase.

```
String nombre = "Juan";
```

Tipos de datos referenciados (objetos)

Arreglos (arrays)

Los arrays permiten almacenar múltiples valores del mismo tipo en una estructura ordenada. Pueden ser de tipos primitivos o referenciados.

```
int[] numeros = {1, 2, 3, 4, 5};
```

```
String[] nombres = {"Ana", "Pedro", "Luis"};
```

Tipos de datos referenciados (objetos)

Clases definidas por el usuario

Puedes crear tus propios tipos de datos definiendo clases. Estas clases se usan para modelar entidades con atributos y comportamientos.

```
public class Persona {
                String nombre;
                int edad;
                public Persona(String nombre, int edad) {
                                this.nombre = nombre;
                                this.edad = edad;
                                }
}
// Uso de la clase
Persona persona1 = new Persona("Carlos", 30);
```

Tipos de datos referenciados (objetos)

Clases Wrapper (envoltorios)

Los tipos primitivos no son objetos en Java. Para tratarlos como objetos, se utilizan las clases wrapper (envoltorios). Cada tipo primitivo tiene una clase asociada en Java. Estas clases se encuentran en el paquete `java.lang` y permiten convertir tipos primitivos en objetos

Tipo primitivo	Clase Wrapper
boolean	Boolean
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double

Operadores

OPERADORES ARITMETICOS DE 2 ENTRADAS		
OPERADOR	**DESCRIPCION**	**FORMATO**
+	Suma	Operando1 + Operando2
-	Resta	Operando1 - Operando2
*	Producto	Operando1 * Operando2
/	Cociente	Operando1 / Operando2
%	Resto (de una división entera)	Operando1 % Operando2

OPERADORES ARITMETICOS DE 1 ENTRADA		
OPERADOR	**DESCRIPCION**	**FORMATO**
-	Cambio de signo	-operando
++	Incremento en uno	++ operando operando ++
--	Decremento en uno	-- operando operando --

Operadores

OPERADORES RELACIONALES		
OPERADOR	**DESCRIPCION**	**FORMATO**
==	Igual	Operando1 == Operando2
!=	Distinto	Operando1 != Operando2
<	Menor	Operando1 < Operando2
<=	Menor o igual	Operando1 <= Operando2
>	Mayor	Operando1 > Operando2
>=	Mayor o igual	Operando1 >= Operando2

OPERADORES LOGICOS DE 2 ENTRADAS		
OPERADOR	**DESCRIPCION**	**FORMATO**
&&	AND (es true el valor de ambos operandos)	Operando1 && Operando2
||	OR (es true uno de los dos operandos)	Operando1 || Operando2

OPERADORES LOGICOS DE 1 ENTRADA		
OPERADOR	**DESCRIPCION**	**FORMATO**
!	NOT (Devuelve true si el operando es false y viceversa	!Operando1

Java y las matemáticas

Java dispone de la clase `Math` para ofrecer al programador todo un conjunto de métodos que implementan funciones matemáticas.

Math.metodo(parámetros)

Java y las matemáticas

```
public class EjemploMath {
        public static void main(String[] args) {        // Valor absoluto
                int valorAbsoluto = Math.abs(-10);
                System.out.println("Valor absoluto de -10: " + valorAbsoluto);
                // Potencia
                double potencia = Math.pow(2, 3);
                System.out.println("2 elevado a la potencia de 3: " + potencia);
                // Raíz cuadrada
                double raiz = Math.sqrt(16);
                System.out.println("Raíz cuadrada de 16: " + raiz);
                // Redondeo hacia arriba
                double redondeoArriba = Math.ceil(4.3);
                System.out.println("Redondeo hacia arriba de 4.3: " + redondeoArriba);
                // Máximo entre dos números
                int maximo = Math.max(5, 10);
                System.out.println("Máximo entre 5 y 10: " + maximo);
                // Número aleatorio entre 0 y 99
                int numeroAleatorio = (int) (Math.random() * 100);
                System.out.println("Número aleatorio entre 0 y 99: " + numeroAleatorio);
                // Trigonometría
                double seno = Math.sin(Math.toRadians(90));
                // Convertir 90 grados a radianes
                System.out.println("Seno de 90 grados: " + seno);
        }
}
```

T4. Introducción a Java

4.1. Java
- 4.1.1. Descripción de la tecnología
- 4.1.2. Versiones de Java
- 4.1.3. Requisitos previos
- 4.1.4. Máquina virtual JVM
- 4.1.5. Entorno de desarrollo JDK
- 4.1.6. Entorno de ejecución JRE
- 4.1.7. Variables de entorno PATH y CLASSPATH
- 4.1.8. Tipos de archivos empleados
- 4.1.9. Recolector de basura

4.2. Entorno de Desarrollo Integrado
- 4.2.1. Características
- 4.2.2. Comparativa IDE para Java
- 4.2.3. Configuración
- 4.2.4. Manejo del IDE

4.3. Elementos de básicos de programación en Java
- 4.3.1. Tipo de datos
- 4.3.2. Operadores
- 4.3.3. Java y las matemáticas

4.4. Estructuras de control en Java
- **4.4.1. Estructuras selectivas (sintaxis)**
- **4.4.2. Estructuras repetitivas**

4.4. Estructuras de control en Java

Estructuras selectivas (alternativa simple)

```
if (expresión que al ser evaluado devuelve un valor boolean)
{
    instrucción 1
    instrucción 2
}
```

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner myObj = new Scanner(System.in);
        System.out.println("Introduce un número: ");
        int iNum1 = myObj.nextInt();
        System.out.println("Introduce otro número: ");
        int iNum2 = myObj.nextInt();
        if (iNum1 == iNum2)
        {
            System.out.println( iNum1 + " es igual que " + iNum2);
        }
}
```

Estructuras selectivas (alternativa doble)

```
if (expresión que al ser evaluado devuelve un valor boolean)
{
    instrucción 1
    instrucción 2
}
else
{
    instrucción 3
    instrucción 4
}
```

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner myObj = new Scanner(System.in);
        System.out.println("Introduce un número: ");
        int iNum1 = myObj.nextInt();
        System.out.println("Introduce otro número: ");
        int iNum2 = myObj.nextInt();
        if (iNum1 == iNum2)
        {
            System.out.println( iNum1 + " es igual que " + iNum2);
        }
        else
        {
            System.out.println( iNum1 + " es distinto que " + iNum2);
        }
    }
}
```

Estructuras selectivas (alternativa múltiple)

```
switch (expresión)
{
    case valor1:
    case valor2:
    case valor3:
    case valor4:
    case valor5:
    default:
}
```

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner myObj = new Scanner(System.in);  // Create a Scanner object
        System.out.println("Dame un día de la semana (1-7): ");
        int iDiaSemana = myObj.nextInt();  // Read user input
        switch (iDiaSemana)
        {
            case 1:
                System.out.println("El dia seleccionado es lunes");
                break;
            case 2:
                System.out.println("El dia seleccionado es martes");
                break;
            case 3:
                System.out.println("El dia seleccionado es miercoles");
                break;
            case 4:
                System.out.println("El dia seleccionado es jueves");
                break;
            case 5:
                System.out.println("El dia seleccionado es viernes");
                break;
            case 6:
                System.out.println("El dia seleccionado es sábado");
                break;
            case 7:
                System.out.println("El dia seleccionado es domingo");
                break;
            default:
                System.out.println("El día de la semana seleccionado no es válido");
                break;
         }
    }
}
```

Estructuras repetitivas (mientras ("while"))

```
while (expresión que al ser evaluado devuelve un valor boolean)
{
    instrucción 1
    instrucción 2
}
```

condición: Una expresión booleana que se evalúa antes de cada iteración. Mientras sea true, el bucle continuará ejecutándose. Si es false, el bucle se detiene.

bloque de código: El código que se ejecutará en cada iteración mientras la condición sea verdadera.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner myObj = new Scanner(System.in);
        System.out.println("¿Cuántos números quieres que escriba? ");
        int iNumero = myObj.nextInt();
        int iContador = 1;
        while (iContador <= iNumero) {
            System.out.println("El resultado es " + iContador);
            iContador++;
        }
    }
}
```

Estructuras repetitivas (hacer-mientras ("do-while"))

```
do
{
    instrucción 1
    instrucción 2
}
while (expresión que al ser evaluado
devuelve un valor boolean)
```

do: Inicia el bloque de código que se ejecutará.

while (condición): Evalúa la condición después de cada ejecución del bloque. Si la condición es true, el bucle continúa. Si es false, el bucle se detiene.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner myObj = new Scanner(System.in);
        System.out.println("¿Cuántos números quieres que escriba? ");
        int iNumero = myObj.nextInt();
        int iContador = 1;
        do {
           System.out.println("El resultado es " + iContador);
           iContador++;
        }
        while (iContador <= iNumero);
    }
}
```

Estructuras repetitivas (desde/para ("for"))

```
for (inicializaciones;condicion;incremento)
{
    instrucción 1
    instrucción 2
}
```

Inicialización: inicializa la variable de control del bucle (generalmente un contador).

Condición: evalúa si el bucle debe continuar o detenerse.

Incremento/decremento: actualiza la variable de control (aumenta o disminuye).

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner myObj = new Scanner(System.in);
        System.out.println("¿Cuántos números quieres que escriba?
");
        int iNumero = myObj.nextInt();
        for (int i = 1; i <= iNumero; i++) {
            System.out.println("El resultado es " + i);
        }
    }
}
```

Grado Superior

Programación

T4. Introducción a Java

Curso 2025/26

Profesor

Raúl Murillo Gallego

«La lógica te llevará de la A a la Z. La imaginación te llevará a cualquier lugar» —Albert Einstein